Esercizio 4

Introduzione
Questo progetto prevede la creazione di un'implementazione di un grafo utilizzando una tabella hash come struttura dati sottostante. L'implementazione supporta sia grafi diretti che non diretti, con etichette opzionali sugli archi.

Implementazione
Il grafo è stato implementato utilizzando l'hash table sia per l'aggiunta di nuovi nodi che di nuovi archi.
Abbiamo creato una nuova struttura per la lista di adiacenze, formata da un hash table contenente i nodi collegati ad un dato nodo, ed il numero di vicini che ha ogni nodo.

Principali funzioni:
graph graph_create - Per creare un nuovo grafo.
graph_add_node - Aggiunge un nodo al grafo
graph_add_edge - Aggiunge un arco al grafo
graph_get_nodes - Restituisce il valore dei nodi del grafo
graph_get_edges - Restituisce gli archi all'interno del grafo
graph_remove_node - Toglie un nodo dal grafo
graph_remove_edge - Toglie un arco dal grafo
graph_free - Libera la memoria allocata per il grafo

Elaborazione del file
L'elaborazione del file prevede di usare la struttura del grafo per visitare in ampiezza i dati contenuti nel file. Il file utilizzato aveva per ogni riga due città e la loro distanza.
Ovviamente le città sono rappresentate dai nodi che andranno a comporre il grafo mentre la distanza tra loro rappresenta l'etichetta dell'arco.

Idea iniziale
La prima idea era quella di utilizzare una lista di archi per creare l'implementazione della lista di adiacenza, ma per fare questo ci sarebbe stato il bisogno di creare un'intera libreria solo per poter gestire l'aggiunta e la rimozione degli archi e la gestione della lista.

Versione finale
Nella versione finale abbiamo optato per usare la libreria già presente dell'hash table per creare la lista di adiacenze.
Questa implementazione occupa meno spazio in memoria, allocando lo spazio solo per i nodi collegati ad un nodo senza dover allocare lo spazio per un'intera struttura dell'arco.

Tempo di esecuzione

Il tempo di esecuzione si aggira intorno ai 0,02 secondi.
Ad esempio, scegliendo “torino” come punto di partenza, la visita impiega 0.023371 secondi

Considerazioni finali

Il progetto ha portato alla realizzazione di una struttura dati grafo versatile e performante. La scelta di utilizzare tabelle hash per gestire sia l'archiviazione dei nodi che le liste di adiacenza si è rivelata ottimale, garantendo elevate prestazioni senza compromettere la leggibilità e la facilità di manutenzione del codice.